Reviewer Pack — Coxeter-Diagram Enumeration Complexity of Boolean Function E…

Entry d42d884eb998 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 00:12:29 UTC

Coxeter-Diagram Enumeration Complexity of Boolean Function Entropy

Entry ID: `d42d884eb998` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 00:12:29 UTC

1. Conjecture

Field A (mathematical branch): Coxeter group theory **Field B** (complexity object): Boolean function entropy

Statement:

> The number of distinct isomorphism classes of Coxeter diagrams for a given boolean function f, denoted as |Diag(f)|, is upper-bounded by the exponential of the Shannon entropy H(f) of f, i.e., |Diag(f)| ≤ exp(H(f)).

Rationale (proposer's reasoning):

> Coxeter group theory provides a rich algebraic structure that can be used to encode combinatorial properties of boolean functions. The connection between Coxeter diagrams and boolean function entropy could potentially reveal new insights into the complexity of boolean functions, as entropy measures the uncertainty or randomness in a system.

Taxonomy category: `COXETER_DIAGRAM_ENUMERATION` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0083d4d317873b81`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a set of randomly generated boolean functions with n variables, if the ratio of |Diag(f)| to exp(H(f)) for all 30 seeds does not exceed 1, support the conjecture.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 4 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Coxeter diagram enumeration" AND "Boolean function entropy"` - `"Shannon entropy" IN COXETER group theory"` - `"isomorphism classes of Coxeter diagrams" AND "upper bound" Boolean functions`

Top relevant hits considered: - [http://arxiv.org/abs/1405.3051v1] Involution Products in Coxeter Groups - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/0804.0947v2] The Dynkin diagram cohomology of finite Coxeter groups - [s2:2403.07989] Boolean intervals in the weak Bruhat order of a finite Coxeter group

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def shannon_entropy(f):
        n = len(f)
        counts = [f.count(0), f.count(1)]
        total = sum(counts)
        if total == 0:
            return 0
        probabilities = [c / total for c in counts]
        entropy = -sum(p * math.log2(p) for p in probabilities if p > 0)
        return entropy

    def coxeter_diagram(f):
        n = len(f)
        diagram = {}
        for i in range(n):
            for j in range(i + 1, n):
                if f[i] != f[j]:
                    key = tuple(sorted([i, j]))
                    if key not in diagram:
                        diagram[key] = 0
                    diagram[key] += 1
        return diagram
```

```
    def is_isomorphic(diag1, diag2):
        if len(diag1) != len(diag2):
            return False
        keys1 = sorted(diag1.keys())
        keys2 = sorted(diag2.keys())
        if keys1 != keys2:
            return False
        for key in keys1:
            if diag1[key] != diag2[key]:
                return False
        return True

    def count_isomorphism_classes(diag):
        classes = []
        for perm in itertools.permutations(range(len(diag))):
            new_diag = {}
            for (i, j), count in diag.items():
                new_key = tuple(sorted([perm[i], perm[j]]))
                if new_key not in new_diag:
                    new_diag[new_key] = 0
                new_diag[new_key] += count
            if all(is_isomorphic(new_diag, cls) for cls in classes):
                classes.append(new_diag)
        return len(classes)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        f = generate_boolean_function(n)
        H_f = shannon_entropy(f)
        diag = coxeter_diagram(f)
        num_classes = count_isomorphism_classes(diag)
        ratio = num_classes / math.exp(H_f)
        results.append({
            "n": n,
            "H_f": H_f,
            "num_classes": num_classes,
            "ratio": ratio
        })

    metric_value = sum(r["ratio"] for r in results) / len(results)
    conjecture_holds = all(r["ratio"] <= 1 for r in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Ratio of |Diag(f)| to exp(H(f))",
        "metric_value": metric_value,
        "instances_tested": len(results),
        "n_max": max(r["n"] for r in results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
```

```
seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means we cannot verify the conjecture's support or falsification. | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18863
2	preregistration	ollama_remote	glm4:latest	0	0	13715
3	novelty	ollama_remote	glm4:latest	0	0	8367
4	novelty	ollama_remote	glm4:latest	0	0	9425
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15660
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9028
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	54494

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27091
9	judge	ollama_remote	glm4:latest	0	0	12051

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 168694 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d42d884eb998.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d42d884eb998.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d42d884eb998.tar.gz` (if generated)